

DS LAB 9

Submitted to: Mam Nayab
Submitted by: Ureen Fatima
BSCS 3 SAP ID: 73897

TASK NO 3

```cpp
#include <iostream>
using namespace std;

const int MAX = 5;
int stackArr[MAX];
int top = -1;

// Push function
void push(int value) {
    if (top == MAX - 1) {
        cout << "Stack overflow!" << endl;
    } else {
        top++;
        stackArr[top] = value;
    }
}

// Pop function
void pop() {
    if (top == -1) {
        cout << "Stack underflow!" << endl;
    } else {
        cout << "Popped: " << stackArr[top] << endl;
        top--;
    }
}

// Search function
int search(int x) {
    int steps = 1;

    for (int i = top; i >= 0; i--) {
        if (stackArr[i] == x) {
            return steps;
        }
        steps++;
    }

    return -1;
}

int main() {
    int value, x;

    // Push five integers
    cout << "Enter 5 integers:" << endl;

    for (int i = 0; i < MAX; i++) {
        cin >> value;
        push(value);
    }

    // Try to push sixth integer
    cout << "Enter sixth integer: ";
    cin >> value;
    push(value);

    // Search for a present value
    cout << "Enter a value to search (present): ";
    cin >> x;

    cout << "Steps from top: " << search(x) << endl;

    // Search for an absent value
    cout << "Enter a value to search (absent): ";
    cin >> x;

    cout << "Steps from top: " << search(x) << endl;

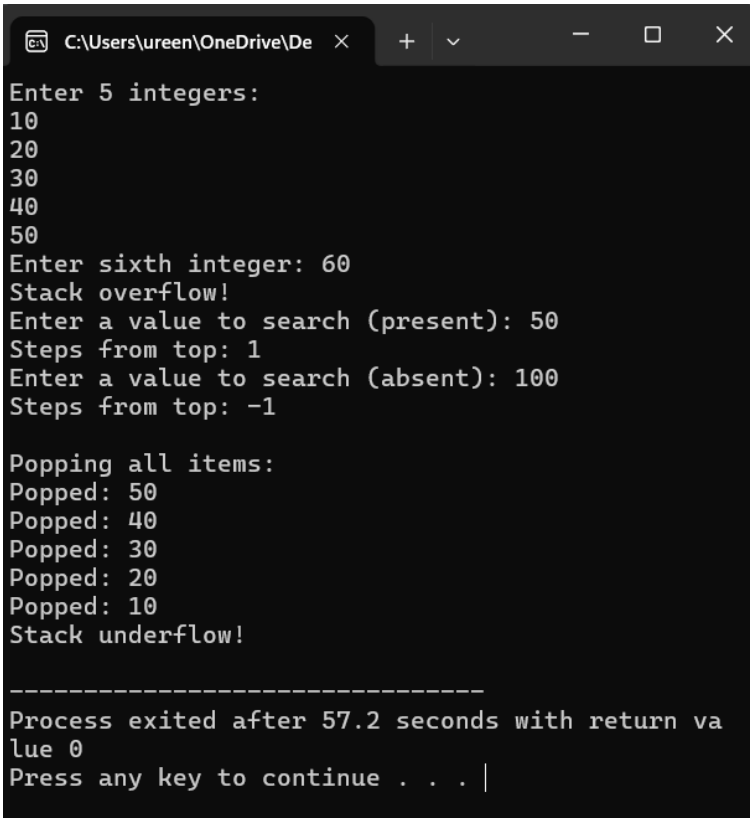
    // Pop all items
    cout << "\nPopping all items:" << endl;

    while (top != -1) {
        pop();
    }

    // Try one extra pop
    pop();

    return 0;
}
```

Output:



Dry Run Example

Suppose you enter these five integers: `10, 20, 30, 40, and 50`.

Operation	Result
Push 10, 20, 30, 40, 50	Stack is full
Push sixth value (60)	`Stack overflow!`
Search 50	Returns `1` (top value)
Search 100	Returns `-1` (not found)
Pop	`Popped: 50`
Pop	`Popped: 40`
Pop	`Popped: 30`
Pop	`Popped: 20`
Pop	`Popped: 10`
Extra pop	`Stack underflow!`

Task 2: Dry Run of (A+B)*C

Expected postfix output: `AB+C*`

Scanned character	Stack (bottom → top)	Postfix output
`(`	`(`	Empty
`A`	`(`	`A`
`+`	`(, +`	`A`
`B`	`(, +`	`AB`
`)`	Empty	`AB+`
`*`	`*`	`AB+`
`C`	`*`	`AB+C`
End (pop `*`)	Empty	`AB+C*`

Explanation:

1. `A` and `B` are added to the output.
2. When `)` appears, `+` is popped and added to the output. The opening bracket is removed.
3. `*` is pushed onto the stack.
4. `C` is added to the output.
5. At the end, `*` is popped.

Final answer: `AB+C*`

TASK NO 1

Rules:

- Operands (A, B, C, D) are directly added to the postfix output.
- `*` and `/` have higher precedence than `+` and `-`.
- `(` is pushed onto the stack.
- `)` means pop operators until `(` is found. Remove `(` without adding it to the output.
- At the end, pop all remaining operators.

1. Expression: `A+B*C`

Scanned character	Stack (bottom → top)	Postfix output
A	Empty	A
+	+	A
B	+	AB

*	+, *	AB
C	+, *	ABC
End	+	ABC*
End	Empty	ABC*+

Answer: `ABC*+`

Why? `*` has higher precedence than `+`, so `B*C` is completed first.

2. Expression: `(A-B)*C`

Scanned character	Stack (bottom → top)	Postfix output
(	(	Empty
A	(	A
-	(, -	A
B	(, -	AB
)	Empty	AB-
*	*	AB-
C	*	AB-C
End	Empty	AB-C*

Answer: `AB-C*`

Why? Parentheses force `A-B` to be completed first. Then the result is multiplied by `C`.

3. Expression: `A*B+C/D`

Scanned character	Stack (bottom → top)	Postfix output
A	Empty	A
*	*	A
B	*	AB
+	+	AB*
C	+	AB*C
/	+, /	AB*C
D	+, /	AB*CD
End	+	AB*CD/
End	Empty	AB*CD/+

Answer: `AB*CD/+`

Why? Multiplication and division have higher precedence than addition, so `A*B` and `C/D` are evaluated first.